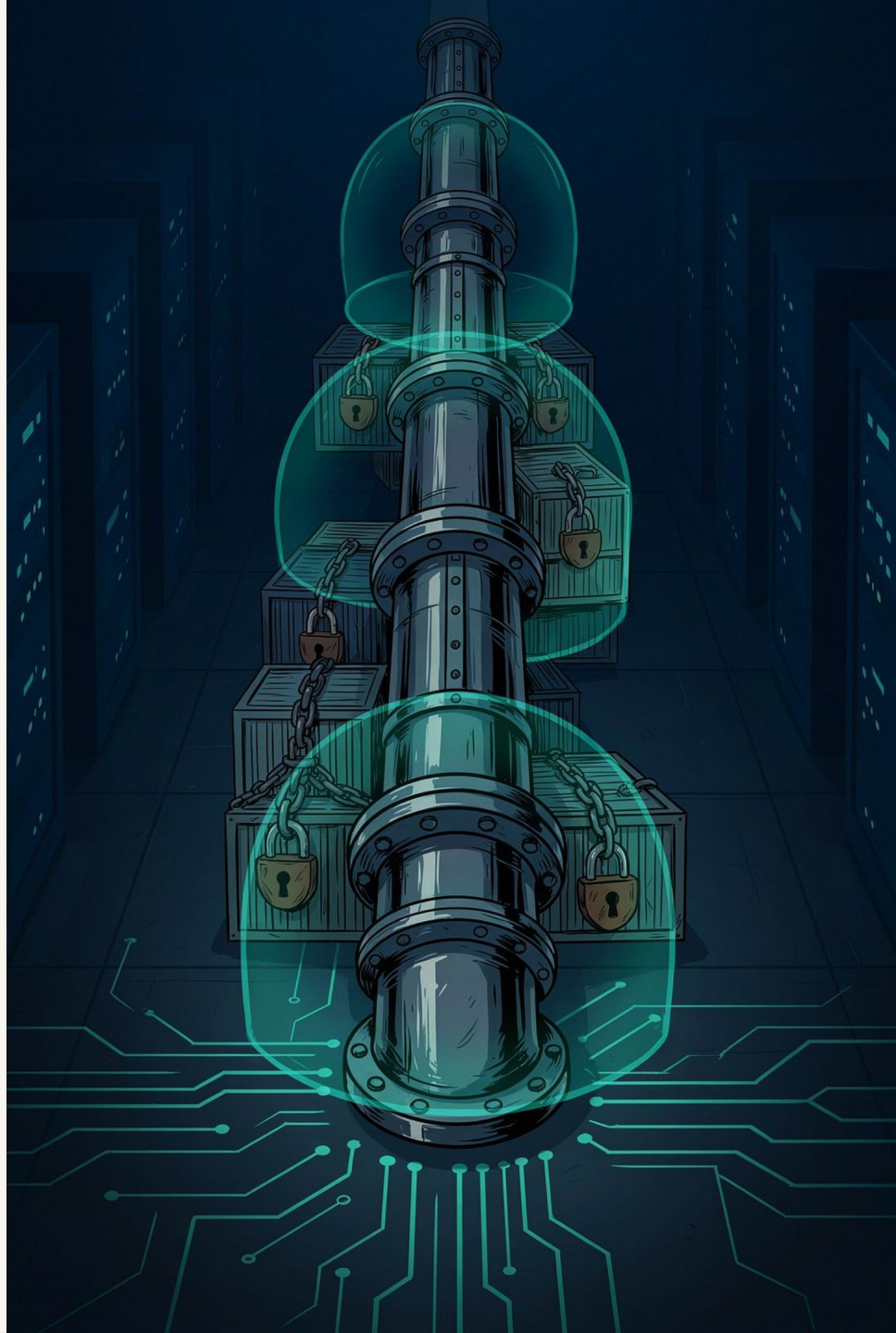
# Module 11 – Supply Chain Security

## Securing Software from Code to Kubernetes

A deep-dive enterprise training module for Kubernetes Administrators, DevOps, DevSecOps, and Platform Engineers — covering the full spectrum of supply chain threats, defenses, and enforcement patterns in cloud-native environments.

4-HOUR DEEP DIVE | ENTERPRISE TRAINING | MODULE 11



# Learning Objectives

By the end of this module, you will be equipped to design, implement, and operate a secure software supply chain in a Kubernetes-native enterprise environment.

## Threat Landscape

Understand modern software supply chain architecture and where attacks occur across the SDLC.

## Image Security

Apply secure image build practices, vulnerability scanning, and CVE triage workflows.

## SBOM

Generate, manage, and consume Software Bills of Materials using CycloneDX and SPDX standards.

## Image Signing

Sign and verify container images using Cosign and the Sigstore ecosystem for cryptographic trust.

## Policy Enforcement

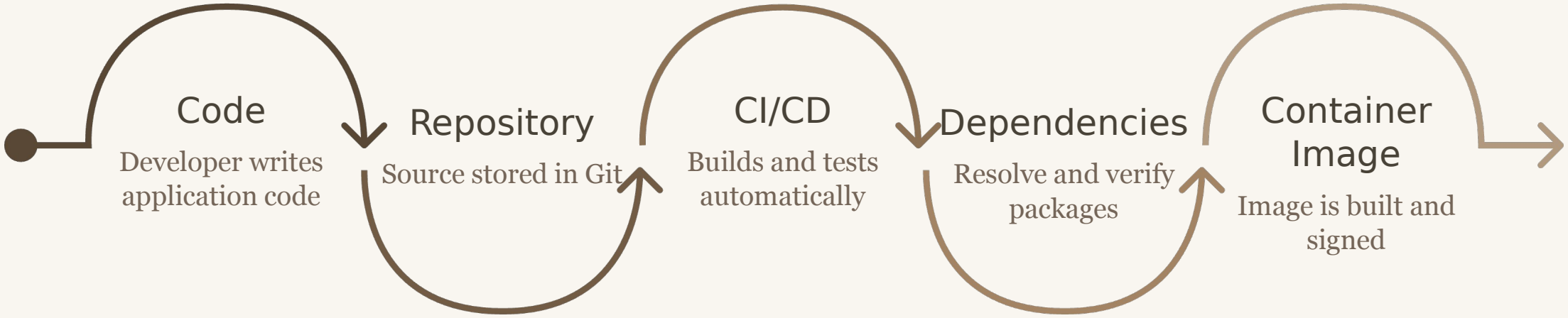
Enforce signature verification and admission policies in Kubernetes using Kyverno and OPA Gatekeeper.

## Enterprise Best Practices

Implement end-to-end DevSecOps pipelines aligned with NIST, CNCF, and government compliance frameworks.

# What is the Software Supply Chain?

Every application you deploy in Kubernetes has traveled a long, vulnerable path before reaching production. The software supply chain encompasses every tool, person, process, and artifact involved in that journey — and **every stage is a potential attack surface**.




⚠ Every stage in this pipeline can be compromised. Security must be enforced continuously, not just at the end.

# Supply Chain Attack Surface

Attackers don't just target your application — they target the weakest link in the chain that delivers it. Below is a breakdown of each pipeline stage and the realistic attack vectors that enterprise teams face.

| Pipeline Stage | Attack Vector | Example Threat |
| --- | --- | --- |
| Developer Laptop | Credential theft, malware | Stolen SSH/GPG keys, compromised IDE plugins |
| Git Repository | Malicious commits, secret leakage | Hardcoded API keys, typosquatted actions |
| Third-party Libraries | Dependency poisoning, typosquatting | Log4Shell, malicious npm packages |
| Build Server | CI environment compromise | Codecov, CircleCI secrets leak |
| Container Image | Image tampering, embedded malware | Crypto miners baked into layers |
| Image Registry | Registry compromise, tag mutation | Overwriting a tag post-push |
| Kubernetes Cluster | Admission bypass, RBAC escalation | Deploying unsigned, unscanned images |
| Production Runtime | Runtime malware, container escape | Fileless malware, privilege escalation |

# Famous Supply Chain Attacks

These real-world incidents reshaped how enterprises think about software provenance, dependency trust, and build pipeline integrity. Each one exposed a different segment of the supply chain.

| Attack | What Happened | Attack Vector | Business Impact | Lesson Learned |
|---|---|---|---|---|
| **SolarWinds (2020)** | Malicious code injected into Orion software build | Compromised build pipeline | 18,000+ orgs breached including US Gov | Sign and verify every build artifact |
| **Log4Shell (2021)** | RCE via JNDI lookup in Log4j 2.x | Vulnerable dependency in millions of apps | Hundreds of millions of endpoints exposed | Maintain real-time SBOM + continuous scanning |
| **Codecov (2021)** | Malicious bash uploader script in CI coverage tool | Tampered CI script exfiltrated env vars | Secrets stolen from thousands of pipelines | Verify checksums of CI tools; use pinned versions |
| **XZ Utils (2024)** | Multi-year backdoor planted in xz compression lib | Social engineering of open-source maintainer | Near-miss: would have compromised SSH on Linux | Audit open-source maintainer trust chains |
| **3CX Attack (2023)** | Trojanized desktop app via poisoned dependency | Upstream compromised library (Trading Technologies) | Widespread enterprise VoIP client compromise | SBOM + binary provenance tracking is essential |
| **CircleCI (2023)** | Session token compromise → CI secrets exfiltrated | Malware on employee laptop | Customers urged to rotate all secrets immediately | Use OIDC short-lived tokens; avoid long-lived secrets |

# Container Image Security

A container image is not a single file — it is a stack of immutable filesystem layers. **Every layer added during the build process can introduce vulnerabilities, exposed secrets, or unnecessary attack surface.** Understanding image anatomy is the prerequisite for securing it.

1

## Base Image

The OS foundation — Ubuntu, Alpine, Distroless. Outdated base images account for the majority of known CVEs.

2

## OS Packages

System binaries, libc, coreutils. Every installed package expands the vulnerability surface.

3

## Runtime Libraries

Language runtimes (JVM, Python, Node). Vulnerable versions are common and frequently exploited.

4

## Application Code

Your source code and compiled artifacts — subject to SAST, SCA, and secrets scanning.

5

## Configuration & Secrets

The most dangerous layer: hardcoded credentials, tokens, and env vars that persist in image history.

# Common Container Vulnerabilities

Container vulnerabilities span insecure configurations, outdated software, and runtime threats. These are the most prevalent issues found in enterprise Kubernetes environments during security audits.

## Weak Base Image

Using `ubuntu:latest` or `debian:latest` pulls in hundreds of unpatched packages without version locking.

## Secrets in Image

API keys, database passwords, and certificates embedded in image layers — visible via `docker history`.

## Unnecessary Tools

Including `curl`, `wget`, shells, or package managers provides attackers post-exploit living-off-the-land capabilities.

## Outdated Packages

Stale OS packages and libraries harbor known CVEs that scanners can detect and report — but only if you run them.

## Running as Root

Containers running as UID 0 can trivially escalate privileges or escape to the host if a container breakout exists.

## Embedded Malware

Crypto miners and backdoors injected via compromised base images or malicious CI pipeline steps — often undetected at runtime.

# Secure Image Build Best Practices

The gap between a vulnerable and a hardened container image often comes down to a few deliberate Dockerfile decisions. These choices dramatically reduce CVE exposure and runtime attack surface.

## Secure Practices

- **Distroless / Alpine** base images — minimal OS surface
- **Multi-stage builds** — strip build tools from final image
- **Non-root user** — `USER 1001` in Dockerfile
- **Version pinning** — `FROM alpine:3.19.1@sha256:...`
- **Minimal packages** — only what the app requires
- **No secrets** — use runtime secret injection (Vault, ESO)
- **Read-only filesystem** — `readOnlyRootFilesystem: true`

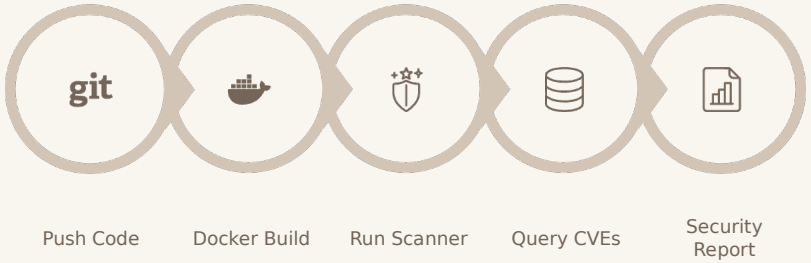
## Insecure Practices

- `FROM ubuntu:latest` — unpinned, bloated
- Single-stage builds — build tools left in final image
- Running as `root` — default in many base images
- `apt install curl wget vim` — unnecessary binaries
- Hardcoded passwords in `ENV` or `ARG`
- `RUN curl https://... | bash` — remote code execution risk
- No `.dockerignore` — leaking source, keys, config files

Multi-stage builds are the single highest-impact Dockerfile change for reducing CVE count in production images.

# Vulnerability Scanning

Image scanning matches every package in your container against known CVE databases (NVD, GHSA, OSV). Integrating scanning into CI/CD gates prevents vulnerable images from ever reaching the registry — or production.




## Severity Levels

| Level | CVSS Score | Recommended Action |
|---|---|---|
| **Critical** | 9.0–10.0 | Block deployment immediately |
| **High** | 7.0–8.9 | Block or require exception |
| **Medium** | 4.0–6.9 | Track and remediate in sprint |
| **Low** | 0.1–3.9 | Log and monitor |

## Enterprise Scanners

### Trivy

CNCF open-source

### Grype

Anchore open-source

### Snyk

Dev-first SaaS

### Prisma Cloud

Palo Alto enterprise

# What is an SBOM?

A **Software Bill of Materials (SBOM)** is a machine-readable inventory of every component inside a software artifact — analogous to a nutrition label on packaged food. The US Executive Order 14028 mandated SBOMs for all software sold to the federal government, accelerating enterprise adoption globally.

1. **Container Image**
   The artifact under inspection — layers, base OS, application
2. **Components Listed**
   Every package, library, dependency, and transitive dependency
3. **Metadata Captured**
   Name, version, license, supplier, hash, and known vulnerabilities
4. **SBOM Document**
   Structured JSON/XML output in CycloneDX or SPDX format — stored alongside image in OCI registry

> SBOMs are not a one-time artifact — they must be regenerated with every image build and stored with full provenance for audits.

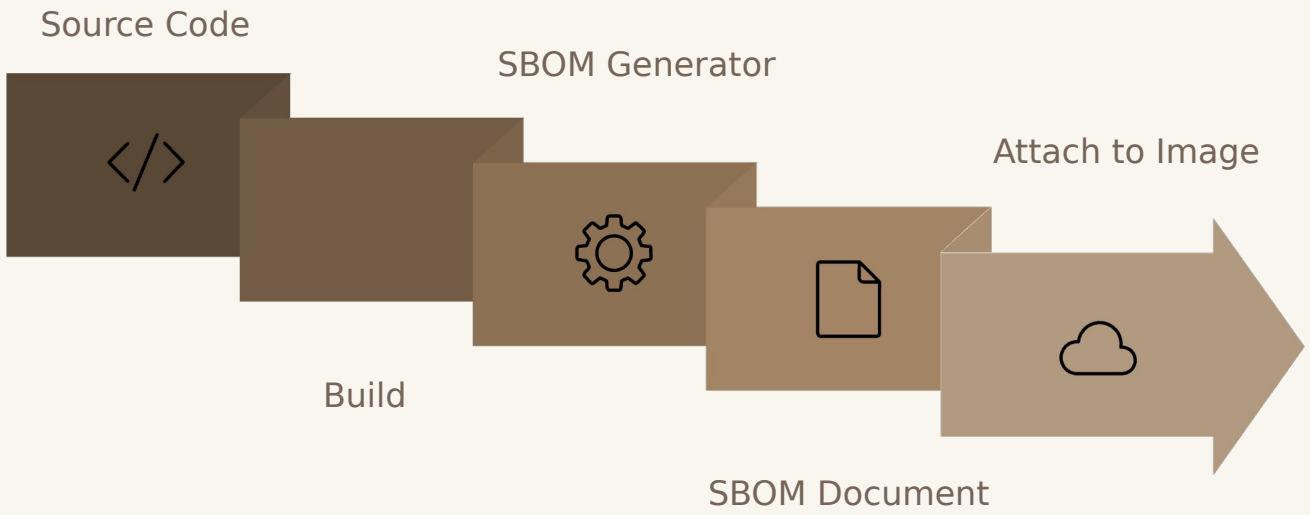
# SBOM Standards Comparison

Three standards dominate the SBOM landscape. Enterprise tool selection should align with your compliance framework, industry vertical, and existing CI/CD toolchain.

| Attribute | CycloneDX | SPDX | SWID |
| --- | --- | --- | --- |
| **Origin** | OWASP | Linux Foundation | ISO/IEC 19770-2 |
| **Primary Purpose** | Security-focused SBOM | License compliance & provenance | Software asset management |
| **Format** | JSON, XML, Protobuf | JSON, YAML, RDF, Tag-Value | XML |
| **Industry Adoption** | High — security tooling standard | High — open-source & legal focus | Moderate — gov/enterprise IT asset mgmt |
| **Key Tools** | Syft, Trivy, Grype, Anchore | Syft, SPDX Tools, FOSSology | NIST tools, enterprise IT management |
| **Kubernetes/OCI** | Native OCI attachment support | OCI-compatible | Limited cloud-native tooling |
| **US Gov Compliance** | NTIA & EO 14028 compliant | NTIA minimum elements compliant | Partial compliance |

# SBOM Generation Workflow

SBOMs should be generated automatically as part of every CI/CD pipeline run — immediately after the image build step — and attached as OCI artifacts to the registry alongside the image digest they describe.




## Open-Source Tools

- **Syft** (Anchore) — generates CycloneDX & SPDX from images, filesystems, and source
- **Trivy** (Aqua) — combined scanner + SBOM generator
- **CycloneDX CLI** — direct CycloneDX generation from build manifests

## Commercial Tools

- **Docker Scout** — integrated into Docker Hub and Docker Desktop
- **Anchore Enterprise** — policy-driven SBOM management at scale
- **Snyk** — developer-first SBOM with remediation guidance

# Why SBOM Matters for Enterprise

An SBOM transforms opaque binary artifacts into transparent, auditable inventories. This unlocks capabilities that were previously manual, slow, or impossible at enterprise scale.

## Incident Response

When a new CVE drops (e.g. Log4Shell), query your SBOM registry to instantly identify every affected image — no manual grep required.

## Dependency Tracking

Maintain a complete graph of direct and transitive dependencies across your entire container portfolio.

## Executive Risk Reporting

Translate technical vulnerability data into business risk metrics — number of affected workloads, severity distribution, remediation SLAs.

## License Compliance

Detect GPL, AGPL, and other copyleft licenses before they create legal obligations in commercial products.

## Government Compliance

Required by US EO 14028, CISA, DoD, and increasingly by EU Cyber Resilience Act for software sold to regulated sectors.

## Software Audits

Satisfy third-party security audits and customer due diligence requests with machine-readable provenance evidence.

# Image Signing — Why It Matters

Container image tags are **mutable by default**. An attacker who gains registry write access can silently overwrite `myapp:latest` with a malicious image — same tag, same apparent source, zero indication of tampering. Image signing breaks this attack by binding a cryptographic proof to the image digest.

### ⚠ Without Image Signing

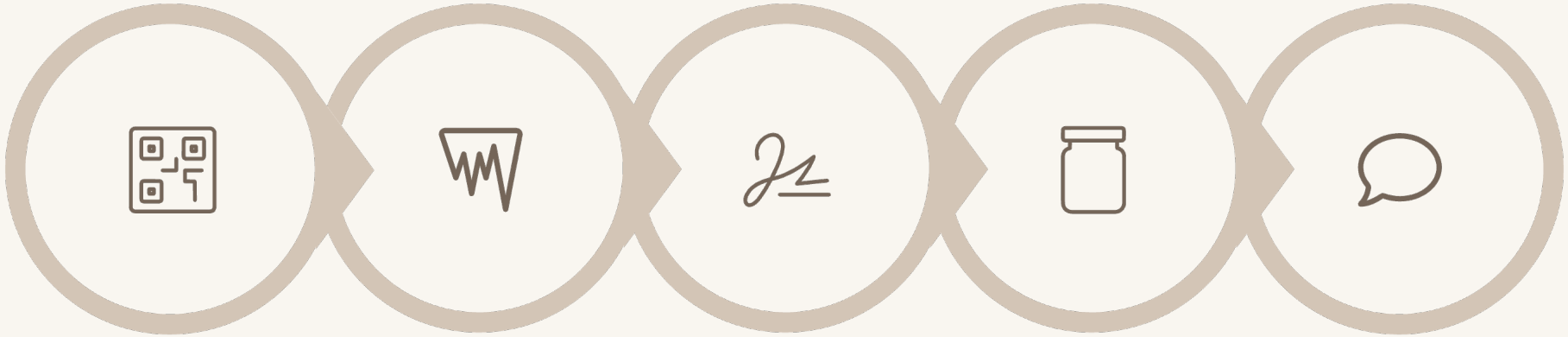
1. Attacker gains registry credentials
2. Overwrites `myapp:v1.2.0` with malicious image
3. Same tag, same apparent name
4. Kubernetes pulls and deploys the tampered image
5. Malware runs in production — undetected

### With Image Signing

1. Developer signs image with private key post-build
2. Signature stored as OCI artifact in registry
3. Admission controller verifies signature at deploy time
4. Signature mismatch → deployment blocked
5. Tampered image never reaches a running Pod

# Digital Signature Workflow

The end-to-end signing and verification workflow integrates into the CI/CD pipeline post-build and is enforced at the Kubernetes admission layer — creating a cryptographic chain of trust from developer commit to running Pod.




### Signing Key

Private key held in KMS (AWS KMS, GCP KMS, HashiCorp Vault) — never stored in CI environment variables.

### OCI Artifact

Signature stored as a separate OCI artifact co-located with the image in the same registry namespace.

### Admission Gate

Kubernetes Admission Controller queries the registry for the signature at deploy time — not at build time.

# Cosign Deep Dive

**Cosign** is the CNCF-graduated tool for signing and verifying OCI container images. It is part of the broader **Sigstore** ecosystem — an open-source PKI-as-a-service initiative backed by Google, Red Hat, and Chainguard that makes software signing accessible to every developer.

## Core Capabilities

### Image Signing

Signs any OCI image or artifact using `cosign sign IMAGE@sha256:...`

### Verification

`cosign verify` validates against a public key or Rekor transparency log entry

### Keyless Signing

Uses short-lived OIDC-bound certificates — no long-lived private keys to manage or rotate

### OIDC Integration

Works natively with GitHub Actions, GitLab CI, Google Workload Identity — ties signature to CI identity

## Sigstore Ecosystem

### Cosign

Image signing & verification

### Rekor

Tamper-proof transparency log for signatures

### Fulcio

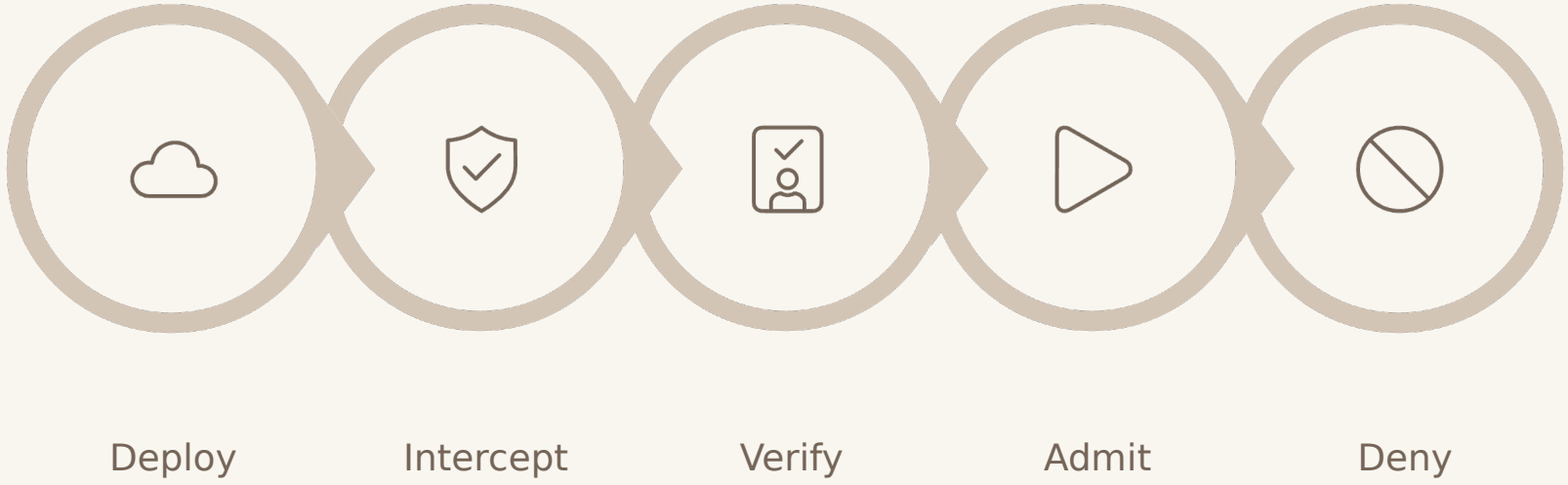
OIDC-based certificate authority — issues short-lived certs

### Policy Controller

Kubernetes admission webhook for Sigstore-based policies

# Kubernetes Signature Verification

Signature verification must be enforced at the Kubernetes admission layer — not just in CI. Without admission-time enforcement, developers or attackers can bypass CI gates and deploy unsigned images directly via `kubectl`.




### Kyverno

Kubernetes-native policy engine. Verify image signatures with a single `ClusterPolicy` using the `verifyImages` rule — no Rego required. Best choice for teams new to policy enforcement.

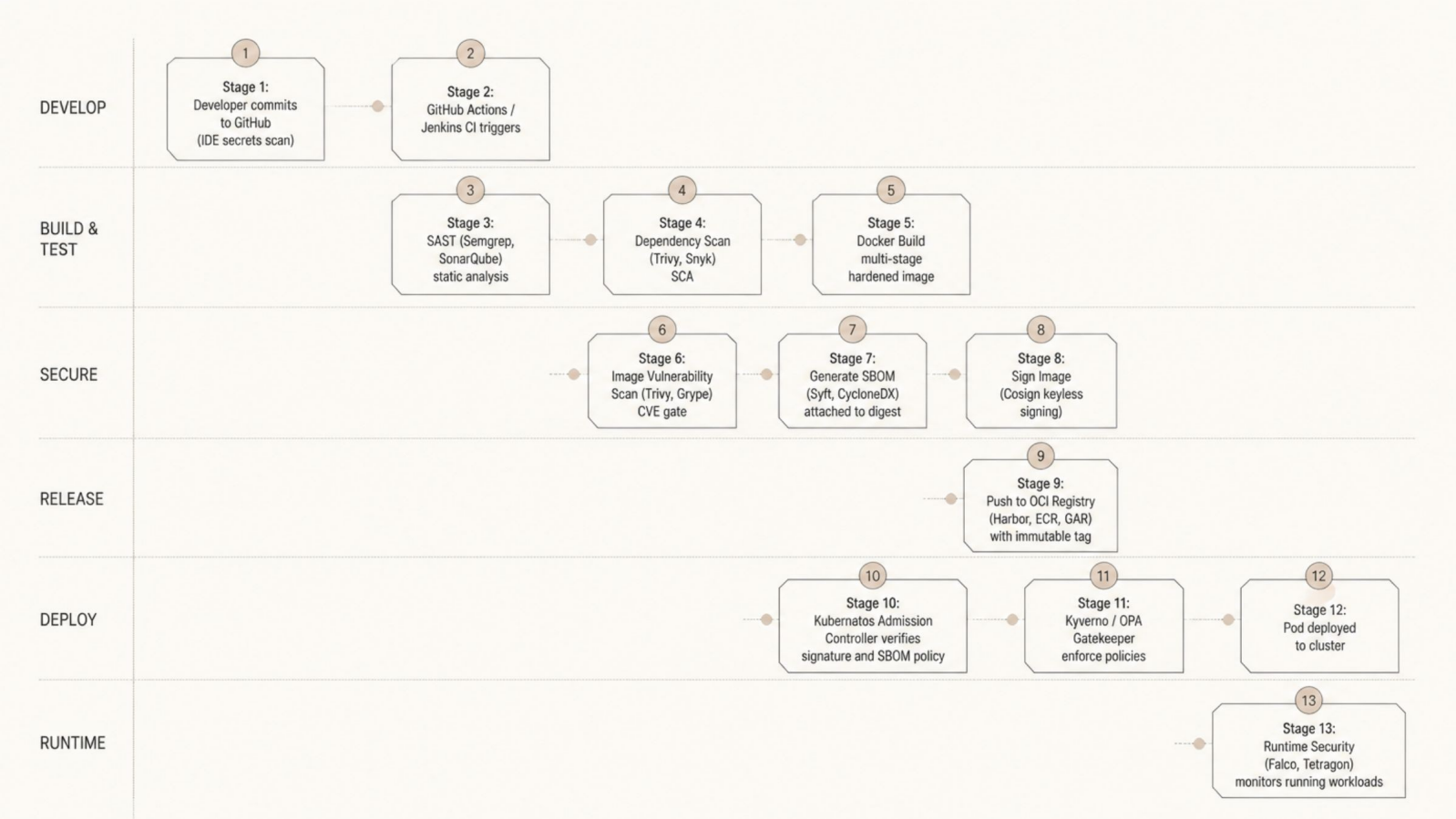
### OPA Gatekeeper

Rego-based policy engine. Highly expressive for complex multi-condition policies. Requires Ratify or Connaisseur as a companion verifier for image signature enforcement.

Ratify (CNCF sandbox) is the recommended OCI artifact verifier for Gatekeeper-based signature enforcement in enterprise environments.

# Enterprise Secure Supply Chain Pipeline

This is the reference DevSecOps pipeline architecture for cloud-native enterprises — integrating every security control from developer commit to production runtime. Each gate is a decision point: fail fast, block early, and enforce continuously.




Every stage produces an attestation artifact (SBOM, scan report, signature) that forms an auditable chain of custody for compliance and incident response.

# Best Practices Checklist

Use this checklist as your supply chain security scorecard. Mature enterprises should have every item enforced as code — not just documented as policy.

## Development & Build

- [x] Use minimal/distroless base images
- [x] Pin image digests, not tags
- [x] Multi-stage builds — no build tools in final image
- [x] Run as non-root user (UID 1000+)
- [x] No secrets in Dockerfile or image layers
- [x] Scan dependencies pre-build (SCA)

## Registry & Artifacts

- [x] Use immutable tags in production
- [x] Sign every image with Cosign
- [x] Generate and attach SBOM to every build
- [x] Enforce registry access controls (RBAC)

## Kubernetes Enforcement

- [x] Enforce image signature verification at admission
- [x] Require SBOM attestation via policy
- [x] Block privileged containers via PodSecurity
- [x] Enforce least-privilege RBAC
- [x] Use read-only root filesystems

## Runtime & Continuous

- [x] Deploy runtime security (Falco / Tetragon)
- [x] Continuous image re-scanning in registry
- [x] Alert on new CVEs against deployed SBOM
- [x] Regular supply chain posture reviews
- [x] Automate policy-as-code in GitOps workflow

# Key Takeaways

# Trust Nothing. Verify Everything. Secure Every Artifact.

### Supply Chain = Full Lifecycle

Security doesn't start at the container — it starts at the developer's workstation and must be enforced at every stage through to production runtime.

### SBOM is the Foundation

You cannot respond to incidents, meet compliance requirements, or manage risk without a real-time, machine-readable inventory of what's running in your cluster.

### Sign and Verify

Image signing with Cosign and admission-time verification via Kyverno is the most impactful control you can implement to prevent tampered images from running in production.

### Shift Left + Enforce Right

Scan early in CI to give developers fast feedback — but always enforce at admission. Defense in depth requires both gates to be active simultaneously.

Supply chain security is now a regulatory requirement — not just a best practice. EO 14028, CISA guidance, and the EU Cyber Resilience Act all mandate provenance, SBOM, and software integrity controls.

# Presentation Design Requirements

This module was designed and delivered to enterprise Kubernetes and DevSecOps teams with the following standards applied throughout every slide. Use these as a reference for extending or customizing this deck for your organization.

1

## Enterprise Diagrams

Architecture, pipeline, and attack chain diagrams replace text-heavy slides. Every workflow is visualized end-to-end with stage labels and decision points.

2

## Professional Icons

Kubernetes, Docker, GitHub, OCI Registry, Cosign, SBOM, CVE, Shield, and Lock icons are used consistently to build visual vocabulary across the deck.

3

## Comparison Tables

Standards (CycloneDX vs SPDX), severity levels, and tool comparisons are always presented in structured tables for rapid comprehension.

4

## Real-World Attacks

All six attack case studies (SolarWinds, Log4Shell, Codecov, XZ Utils, 3CX, CircleCI) include attack vector, business impact, and actionable lessons learned.

5

## DevSecOps Pipeline

The enterprise secure supply chain pipeline (Slide 18) serves as the capstone architecture — every preceding slide teaches a component of that pipeline.

# Module 11 — Lab Exercises

Hands-on labs reinforce every concept from this module. Each lab is designed to run in a local `kind` cluster or enterprise dev environment and produces real artifacts (SBOMs, signatures, policies) that students can inspect and validate.

## 1 Lab 1: Scan an Image

Run Trivy against a deliberately vulnerable image. Identify Critical and High CVEs. Set a severity gate that would block the image in CI.

## 2 Lab 2: Generate an SBOM

Use Syft to generate a CycloneDX SBOM from a container image. Inspect the output. Push the SBOM as an OCI artifact alongside the image digest.

## 3 Lab 3: Sign with Cosign

Sign a locally built image using Cosign keyless mode with GitHub Actions OIDC. Verify the signature and inspect the Rekor transparency log entry.

## 4 Lab 4: Enforce with Kyverno

Deploy a Kyverno `ClusterPolicy` that blocks unsigned images. Attempt to deploy a signed and unsigned image — observe admission allow/deny behavior.

## 5 Lab 5: Full Pipeline

Chain all labs: build → scan → generate SBOM → sign → push → deploy with Kyverno enforcement. Simulate a tampered image and verify the pipeline blocks it.

# Next Steps & Resources

Continue Your Supply Chain Security Journey

## Standards & Frameworks

- **SLSA (Supply chain Levels for Software Artifacts)** — Google/OpenSSF framework for supply chain integrity levels 1–4
- **NIST SSDF (SP 800-218)** — Secure Software Development Framework
- **CNCF Supply Chain Security Whitepaper** — cloud-native reference architecture
- **OpenSSF Scorecard** — automated open-source project security scoring

## Tools to Explore

- **Sigstore / Cosign** — `sigstore.dev`
- **Syft + Grype** — `anchore.com/open-source`
- **Trivy** — `aquasecurity.github.io/trivy`
- **Kyverno** — `kyverno.io`
- **Falco** — `falco.org` (runtime security)
- **SLSA Verifier** — `github.com/slsa-framework`

## Up Next

**Module 12 – Runtime Security & Threat Detection** — Falco, eBPF, Tetragon, and responding to live cluster intrusions.